Academy Hub — Sesión 7

PostgreSQL, ORM avanzado, constraints, transacciones y optimización

> **Nota:** esta versión corrige los problemas de concreción del documento anterior. Cada sección indica:
>
> 1. Archivo concreto.
> 2. Bloque de código completo (no fragmentos).
> 3. Qué reemplaza.
> 4. Por qué.
> 5. A qué otras partes aplica.

Índice

1. Objetivo general
2. Instalación de PostgreSQL
3. Creación de usuario y base de datos
4. Conexión de Django con PostgreSQL
5. Reset de SQLite y migraciones
6. Constraints: `Meta` completo por modelo
7. Índices: `Meta` completo por modelo
8. ORM avanzado: dónde y por qué
9. Optimización de consultas: tabla por app
10. Transacciones: qué vistas y por qué
11. Servicios: creación de archivos y tests
12. Reorganización de tests por app
13. Backup básico
14. Checklist global
15. Registro de sesión
16. Commit recomendado

---

1. Objetivo general

Hasta ahora Academy Hub funciona sobre:

```
Django
    ↓
SQLite
```

Ahora pasamos a:

```
Django
    ↓
```

```
psycopg
    ↓
PostgreSQL
```

Y aplicamos:

- constraints de base de datos;
- índices simples y compuestos;
- ORM avanzado (`annotate`, `aggregate`, `values`, `values_list`);
- optimización (`select_related`, `prefetch_related`, `Prefetch`);
- transacciones (`transaction.atomic`, `select_for_update`);
- tests de integridad.

---

2. Instalación de PostgreSQL

2.1. Comprobar instalación

```
psql --version
```

Si no está instalado, descargar desde:

```
https://www.enterprisedb.com/downloads/postgres-postgresql-downloads
```

2.2. Añadir `psql` al PATH de Windows

1. `Win + R` → `sysdm.cpl` → Enter.
2. Opciones avanzadas → Variables de entorno.
3. En Variables de usuario → `Path` → Editar → Nuevo.
4. Añadir:

```
C:\Program Files\PostgreSQL\18\bin
```

5. Aceptar todo y cerrar.
6. Cerrar y reabrir CMD o PowerShell.

Comprobar:

```
psql --version
where psql
```

2.3. Puerto

En este documento se usa `5432` como valor por defecto. Si la instalación usa otro puerto (por ejemplo `5433`), se cambia en `.env`.

---

3. Creación de usuario y base de datos

3.1. Entrar a PostgreSQL

```
psql -U postgres -h localhost -p 5432
```

3.2. Crear usuario

```
CREATE USER academyhub_user WITH PASSWORD 'academyhub_password';
```

En producción, contraseña fuerte y fuera de Git.

3.3. Crear base de datos

```
CREATE DATABASE academyhub OWNER academyhub_user;
```

3.4. Comprobar

Salir con `\q` y volver a entrar:

```
psql -U academyhub_user -d academyhub -h localhost -p 5432
```

Dentro:

```
\dt
```

Inicialmente vacía. Correcto.

---

4. Conexión de Django con PostgreSQL

4.1. Instalar driver

```
pip install "psycopg[binary]"
pip freeze > requirements.txt
```

4.2. `.env`

Archivo: `.env` (raíz del proyecto).

Qué reemplaza: se añaden estas líneas al final del `.env` existente. No se borra nada de lo que ya hay.

```
POSTGRES_DB=academyhub
POSTGRES_USER=academyhub_user
POSTGRES_PASSWORD=academyhub_password
POSTGRES_HOST=localhost
POSTGRES_PORT=5432
POSTGRES_TEST_DB=academyhub_test
```

4.3. `.env.example`

Archivo: `.env.example` (raíz del proyecto).

Qué reemplaza: se añaden estas líneas al final del `.env.example` existente.

```
POSTGRES_DB=
POSTGRES_USER=
POSTGRES_PASSWORD=
POSTGRES_HOST=localhost
POSTGRES_PORT=5432
POSTGRES_TEST_DB=
```

Recordatorio:

- `.env` NO se sube a Git.
- `.env.example` SÍ se sube.

4.4. `config/settings/development.py`

Archivo: `config/settings/development.py`.

Qué reemplaza: el archivo entero.

```
import os

from .base import *

DEBUG = True

DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.postgresql",
        "NAME": os.getenv("POSTGRES_DB", "academyhub"),
        "USER": os.getenv("POSTGRES_USER", "academyhub_user"),
        "PASSWORD": os.getenv("POSTGRES_PASSWORD", ""),
        "HOST": os.getenv("POSTGRES_HOST", "localhost"),
        "PORT": os.getenv("POSTGRES_PORT", "5432"),
    }
}
```

4.5. `config/settings/production.py`

Archivo: `config/settings/production.py`.

Qué reemplaza: el archivo entero.

```
import os

from .base import *

DEBUG = False

ALLOWED_HOSTS = os.getenv("ALLOWED_HOSTS", "").split(",")

DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.postgresql",
        "NAME": os.getenv("POSTGRES_DB"),
        "USER": os.getenv("POSTGRES_USER"),
        "PASSWORD": os.getenv("POSTGRES_PASSWORD"),
```

```
        "HOST": os.getenv("POSTGRES_HOST"),
        "PORT": os.getenv("POSTGRES_PORT", "5432"),
    }
}
```

4.6. `config/settings/testing.py`

Archivo: `config/settings/testing.py`.

Qué reemplaza: el archivo entero.

```
import os

from .base import *

DEBUG = False

DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.postgresql",
        "NAME": os.getenv("POSTGRES_TEST_DB", "academyhub_test"),
        "USER": os.getenv("POSTGRES_USER", "academyhub_user"),
        "PASSWORD": os.getenv("POSTGRES_PASSWORD", ""),
        "HOST": os.getenv("POSTGRES_HOST", "localhost"),
        "PORT": os.getenv("POSTGRES_PORT", "5432"),
        "TEST": {
            "NAME": "test_academyhub",
        },
    }
}
```

4.7. Probar conexión

```
python manage.py check
```

Si no aparece error de conexión, está bien.

---

5. Reset de SQLite y migraciones

5.1. Borrar SQLite

Windows PowerShell:

```
Remove-Item db.sqlite3
```

5.2. Borrar migraciones de apps propias

```
Get-ChildItem accounts\migrations, academies\migrations, courses\migrations, enrollments\migrations, core\migrations,
```

5.3. Borrar `__pycache__`

```
Get-ChildItem -Recurse -Directory -Filter "__pycache__" | Remove-Item -Recurse -Force
```

5.4. Crear migraciones y aplicarlas

```
python manage.py makemigrations
python manage.py migrate
```

5.5. Crear superusuario

```
python manage.py createsuperuser
```

5.6. Comprobar en PostgreSQL

```
psql -U academyhub_user -d academyhub -h localhost -p 5432
```

Dentro:

```
\dt
SELECT id, username, email FROM accounts_user;
\q
```

---

6. Constraints: `Meta` completo por modelo

Regla: en cada modelo, se reemplaza **el** `Meta` **entero** por el que se muestra. No se añade solo el bloque `constraints`, porque eso dejaría el `Meta` incompleto.

6.1. `accounts/models.py` — `User`

`User` ya tiene `email` único. **No se toca el** `Meta` de `User`. Se mantiene tal cual está.

6.2. `academies/models.py` — `Academy`

Qué reemplaza: el `class Meta` actual de `Academy`.

```
class Meta:
    verbose_name = "Academia"
    verbose_name_plural = "Academias"
    ordering = ["-created_at"]
    constraints = [
        models.CheckConstraint(
            condition=models.Q(views_count__gte=0),
            name="academy_views_count_gte_zero",
        ),
    ]
    indexes = [
        models.Index(fields=["city"]),
        models.Index(fields=["is_active"]),
        models.Index(fields=["is_verified"]),
```

```
        models.Index(fields=["is_active", "-created_at"]),
        models.Index(fields=["city", "is_active"]),
        models.Index(fields=["-views_count"], name="academy_views_idx"),
    ]
```

Por qué: `views_count` nunca debe ser negativo. Los índices nuevos sirven para filtros y ordenaciones de la Sesión 6.

6.3. `academies/models.py` — `AcademyFavorite`

Qué reemplaza: el `class Meta` actual.

```
class Meta:
    ordering = ["-created_at"]
    constraints = [
        models.UniqueConstraint(
            fields=["user", "academy"],
            name="unique_user_academy_favorite",
        ),
    ]
    indexes = [
        models.Index(fields=["user", "-created_at"]),
        models.Index(fields=["academy"]),
    ]
```

Por qué: garantiza que un usuario no repita favorito. Índices para listar favoritos por usuario y por academia.

6.4. `academies/models.py` — `Teacher`

Qué reemplaza: el `class Meta` actual.

```
class Meta:
    verbose_name = "Profesor"
    verbose_name_plural = "Profesores"
    ordering = ["last_name", "first_name"]
    indexes = [
        models.Index(fields=["academy"]),
        models.Index(fields=["is_active"]),
        models.Index(fields=["academy", "is_active"]),
    ]
```

Por qué: no aplica constraint específica. Índices para listar profesores por academia.

6.5. `courses/models.py` — `Course`

Qué reemplaza: el `class Meta` actual de `Course`.

```
class Meta:
    verbose_name = "Curso"
    verbose_name_plural = "Cursos"
    ordering = ["-created_at"]
    constraints = [
```

```
        models.CheckConstraint(
            condition=models.Q(views_count__gte=0),
            name="course_views_count_gte_zero",
        ),
        models.CheckConstraint(
            condition=models.Q(duration_hours__gte=0),
            name="course_duration_hours_gte_zero",
        ),
    ]
    indexes = [
        models.Index(fields=["status"]),
        models.Index(fields=["created_at"]),
        models.Index(fields=["level"]),
        models.Index(fields=["modality"]),
        models.Index(fields=["category"]),
        models.Index(fields=["academy"]),
        models.Index(fields=["status", "-created_at"]),
        models.Index(fields=["status", "category"]),
        models.Index(fields=["status", "-views_count"]),
    ]
```

Por qué: `views_count` y `duration_hours` nunca negativos. Índices para filtros y ordenación de la Sesión 6.

6.6. `courses/models.py` — `Convocatoria`

Qué reemplaza: el `class Meta` actual de `Convocatoria`.

```
class Meta:
    verbose_name = "Convocatoria"
    verbose_name_plural = "Convocatorias"
    ordering = ["start_date"]
    constraints = [
        models.CheckConstraint(
            condition=models.Q(price__gt=0),
            name="convocatoria_price_gt_zero",
        ),
        models.CheckConstraint(
            condition=models.Q(deposit__gte=0),
            name="convocatoria_deposit_gte_zero",
        ),
        models.CheckConstraint(
            condition=models.Q(reserved_places__gte=0),
            name="convocatoria_reserved_places_gte_zero",
        ),
        models.CheckConstraint(
            condition=models.Q(enrolled_places__gte=0),
            name="convocatoria_enrolled_places_gte_zero",
        ),
    ]
    indexes = [
        models.Index(fields=["start_date"]),
        models.Index(fields=["is_active"]),
        models.Index(fields=["course"]),
        models.Index(fields=["course", "is_active"]),
        models.Index(fields=["start_date", "is_active"]),
    ]
```

Por qué: el precio nunca puede ser 0 o negativo. `deposit`, `reserved_places` y `enrolled_places` nunca negativos. Índices para listados por curso y por fecha.

Nota: no se añade la constraint `total_places >= reserved_places + enrolled_places` porque en desarrollo puede ser problemática si se meten datos a mano. Se valida en vistas.

6.7. `courses/models.py` — `CourseFavorite`

Qué reemplaza: el `class Meta` actual.

```
class Meta:
    ordering = ["-created_at"]
    constraints = [
        models.UniqueConstraint(
            fields=["user", "course"],
            name="unique_user_course_favorite",
        ),
    ]
    indexes = [
        models.Index(fields=["user", "-created_at"]),
        models.Index(fields=["course"]),
    ]
```

Por qué: mismo patrón que `AcademyFavorite`.

6.8. `enrollments/models.py` — `Enrollment`

Qué reemplaza: el `class Meta` actual de `Enrollment`.

```
class Meta:
    verbose_name = "Inscripción"
    verbose_name_plural = "Inscripciones"
    ordering = ["-created_at"]
    constraints = [
        models.UniqueConstraint(
            fields=["student", "convocatoria"],
            name="unique_student_convocatoria",
        ),
        models.CheckConstraint(
            condition=models.Q(amount_paid__gte=0),
            name="enrollment_amount_paid_gte_zero",
        ),
        models.CheckConstraint(
            condition=models.Q(discount_amount__gte=0),
            name="enrollment_discount_amount_gte_zero",
        ),
    ]
    indexes = [
        models.Index(fields=["status"]),
        models.Index(fields=["payment_status"]),
        models.Index(fields=["student", "status"]),
        models.Index(fields=["convocatoria", "status"]),
        models.Index(fields=["student", "-created_at"]),
    ]
```

Por qué: un alumno no puede inscribirse dos veces en la misma convocatoria. Importes nunca negativos. Índices para listados por estado.

6.9. `enrollments/models.py` — `WaitlistEntry`

Qué reemplaza: el `class Meta` actual de `WaitlistEntry`.

```
class Meta:
    verbose_name = "Lista de espera"
    verbose_name_plural = "Listas de espera"
    ordering = ["position", "created_at"]
    constraints = [
        models.UniqueConstraint(
            fields=["student", "convocatoria"],
            name="unique_waitlist_student_convocatoria",
        ),
        models.CheckConstraint(
            condition=models.Q(position__gte=0),
            name="waitlist_position_gte_zero",
        ),
    ]
    indexes = [
        models.Index(fields=["convocatoria", "position"]),
        models.Index(fields=["student"]),
    ]
```

Por qué: un alumno no puede estar dos veces en la lista. Posición nunca negativa. Índices para ordenar la lista por posición.

6.10. `reviews/models.py` — `CourseReview`

Qué reemplaza: el `class Meta` actual.

```
class Meta:
    ordering = ["-created_at"]
    constraints = [
        models.UniqueConstraint(
            fields=["student", "course"],
            name="unique_student_course_review",
        ),
        models.CheckConstraint(
            condition=models.Q(rating__gte=1, rating__lte=5),
            name="coursereview_rating_between_1_and_5",
        ),
    ]
    indexes = [
        models.Index(fields=["course"]),
        models.Index(fields=["student"]),
        models.Index(fields=["course", "-created_at"]),
    ]
```

Por qué: un alumno no puede valorar dos veces el mismo curso. Puntuación entre 1 y 5.

6.11. `reviews/models.py` — `AcademyReview`

Qué reemplaza: el `class Meta` actual.

```
class Meta:
    ordering = ["-created_at"]
    constraints = [
        models.UniqueConstraint(
            fields=["student", "academy"],
            name="unique_student_academy_review",
        ),
        models.CheckConstraint(
            condition=models.Q(rating__gte=1, rating__lte=5),
            name="academyreview_rating_between_1_and_5",
        ),
    ]
    indexes = [
        models.Index(fields=["academy"]),
        models.Index(fields=["student"]),
        models.Index(fields=["academy", "-created_at"]),
    ]
```

6.12. `payments/models.py` — `Discount`

Qué reemplaza: el `class Meta` actual de `Discount` (si no tiene, se añade).

```
class Meta:
    ordering = ["-created_at"]
    constraints = [
        models.CheckConstraint(
            condition=models.Q(value__gte=0),
            name="discount_value_gte_zero",
        ),
    ]
```

6.13. `payments/models.py` — `Payment`

Qué reemplaza: el `class Meta` actual de `Payment`.

```
class Meta:
    ordering = ["-created_at"]
    constraints = [
        models.CheckConstraint(
            condition=models.Q(amount__gte=0),
            name="payment_amount_gte_zero",
        ),
    ]
    indexes = [
        models.Index(fields=["enrollment", "status"]),
        models.Index(fields=["status"]),
    ]
```

6.14. `payments/models.py` — `Scholarship`

Qué reemplaza: el `class Meta` actual de `Scholarship`.

```
class Meta:
    ordering = ["-created_at"]
    constraints = [
        models.CheckConstraint(
            condition=models.Q(amount__gte=0),
            name="scholarship_amount_gte_zero",
        ),
    ]
```

6.15. `talent/models.py` — `PassportEntry`

Qué reemplaza: el `class Meta` actual.

```
class Meta:
    ordering = ["-obtained_at"]
    constraints = [
        models.UniqueConstraint(
            fields=["passport", "competence"],
            name="unique_passport_competence",
        ),
        models.CheckConstraint(
            condition=models.Q(level__gte=1, level__lte=5),
            name="passportentry_level_between_1_and_5",
        ),
    ]
    indexes = [
        models.Index(fields=["passport", "-obtained_at"]),
    ]
```

6.16. `level_tests/models.py` — `LevelTest`

Qué reemplaza: el `class Meta` actual de `LevelTest` (si no tiene, se añade).

```
class Meta:
    ordering = ["course", "title"]
    constraints = [
        models.CheckConstraint(
            condition=models.Q(passing_score__gte=0, passing_score__lte=100),
            name="leveltest_passing_score_between_0_and_100",
        ),
    ]
```

6.17. `level_tests/models.py` — `LevelTestAttempt`

Qué reemplaza: el `class Meta` actual de `LevelTestAttempt`.

```
class Meta:
    ordering = ["-started_at"]
    constraints = [
        models.CheckConstraint(
            condition=models.Q(score__gte=0, score__lte=100),
```

```
            name="leveltestattempt_score_between_0_and_100",
        ),
    ]
    indexes = [
        models.Index(fields=["student", "-started_at"]),
        models.Index(fields=["level_test"]),
    ]
```

6.18. `notifications/models.py` — `Notification`

Qué reemplaza: el `class Meta` actual.

```
class Meta:
    ordering = ["-created_at"]
    indexes = [
        models.Index(fields=["user", "is_read"]),
        models.Index(fields=["user", "is_read", "-created_at"]),
    ]
```

6.19. `evaluations/models.py` — `Evaluation`

Qué reemplaza: el `class Meta` actual.

```
class Meta:
    ordering = ["-created_at"]
    constraints = [
        models.UniqueConstraint(
            fields=["enrollment", "kind"],
            name="unique_evaluation_per_enrollment_kind",
        ),
    ]
    indexes = [
        models.Index(fields=["enrollment", "kind"]),
    ]
```

6.20. Modelos sin cambios

- `accounts.User`: ya tiene `email` único y `Meta` correcto.
- `core.ContactMessage`: `Meta` correcto.
- `enrollments.EnrollmentAnswer`: sin constraints relevantes.
- `enrollments.EnrollmentDocument`: sin constraints relevantes.
- `talent.Competence`: sin constraints relevantes.
- `talent.TalentPassport`: sin constraints relevantes.
- `level_tests.LevelTestQuestion`: sin constraints relevantes.

6.21. Aplicar migraciones

```
python manage.py makemigrations
python manage.py migrate
```

7. ORM avanzado: dónde y por qué

7.1. Regla

> `annotate` solo en listados donde el template muestre un contador o media por fila. `aggregate` solo en detalle o dashboard donde el template muestre un dato global.

Si no se muestra en pantalla, no se calcula. Cada `annotate` o `aggregate` es un `JOIN` + `GROUP BY` adicional en SQL.

7.2. Tabla de aplicación por vista

App	Vista	Acción
`courses`	`CourseListView`	`annotate(favorite_count, reviews_avg)` + `Prefetch` de imágenes
`courses`	`CourseDetailView`	`aggregate` en `get_context_data` (reviews_stats, price_range)
`courses`	`CategoryListView`	`annotate(published_courses_count)`
`courses`	`MyCourseListView`	`annotate(favorite_count)`
`courses`	Resto	Sin cambios
`academies`	`AcademyListView`	`annotate(courses_count, teachers_count, favorite_count, reviews_avg)`
`academies`	`AcademyDetailView`	`aggregate` en `get_context_data` (reviews_stats, courses_count)
`academies`	`TeacherListView`	`annotate(courses_count)`
`academies`	`AcademyFavoriteListView`	Sin cambios
`academies`	Resto	Sin cambios
`level_tests`	`LevelTestListView`	`annotate(attempts_count)`
`level_tests`	Resto	Sin cambios
`dashboard`	`DashboardView`	`aggregate` en `get_context_data`
Resto de apps	Todas	Sin cambios

7.3. `courses/views.py` — `CourseListView`

Qué reemplaza: el método `get_queryset()` actual.

```
from django.db.models import Avg, Count, Prefetch

from .models import CourseImage

class CourseListView(ListView):
```

```
    model = Course
    template_name = "courses/course_list.html"
    context_object_name = "courses"
    paginate_by = 12

    def get_queryset(self):
        queryset = (
            Course.objects
            .filter(status=CourseStatus.PUBLISHED)
            .select_related("academy", "category", "teacher")
            .prefetch_related(
                Prefetch(
                    "images",
                    queryset=CourseImage.objects.order_by("position"),
                )
            )
            .annotate(
                favorite_count=Count("favorites", distinct=True),
                reviews_avg=Avg("reviews__rating"),
            )
        )
        # filtros que ya tenías
        return queryset.distinct()
```

Template `courses/course_list.html` (añadir en cada tarjeta):

```
<p>{{ course.favorite_count }} personas lo han guardado</p>
{% if course.reviews_avg %}
    <p>Valoración media: {{ course.reviews_avg|floatformat:1 }}/5</p>
{% endif %}
```

7.4. `courses/views.py` — `CourseDetailView`

Qué reemplaza: el método `get_context_data()` actual.

```
from django.db.models import Avg, Count, F, Max, Min

class CourseDetailView(DetailView):
    model = Course
    template_name = "courses/course_detail.html"
    context_object_name = "course"

    # get_queryset y get_object igual que antes

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)

        context["is_favorite"] = False
        if self.request.user.is_authenticated:
            context["is_favorite"] = CourseFavorite.objects.filter(
                user=self.request.user,
                course=self.object,
            ).exists()

        context["favorite_count"] = self.object.favorites.count()

        context["reviews_stats"] = self.object.reviews.aggregate(
            avg=Avg("rating"),
            total=Count("id"),
```

```
        )

        context["price_range"] = self.object.convocatorias.filter(
            is_active=True
        ).aggregate(
            min_price=Min("price"),
            max_price=Max("price"),
        )

        return context
```

Template `courses/course_detail.html` (añadir):

```
<p>{{ favorite_count }} personas han guardado este curso.</p>

<p>
    Valoración media:
    {% if reviews_stats.avg %}
        {{ reviews_stats.avg|floatformat:1 }}/5 ({{ reviews_stats.total }} reseñas)
    {% else %}
        Sin reseñas todavía
    {% endif %}
</p>

{% if price_range.min_price %}
    <p>Desde {{ price_range.min_price }} € hasta {{ price_range.max_price }} €</p>
{% endif %}
```

7.5. `courses/views.py` — `CategoryListView`

Qué reemplaza: el método `get_queryset()` actual.

```
from django.db.models import Count, Q

class CategoryListView(ListView):
    model = Category
    template_name = "courses/category_list.html"
    context_object_name = "categories"

    def get_queryset(self):
        return (
            Category.objects
            .filter(is_active=True)
            .annotate(
                published_courses_count=Count(
                    "courses",
                    filter=Q(courses__status=CourseStatus.PUBLISHED),
                    distinct=True,
                ),
            )
        )
```

Template `courses/category_list.html`:

```
<p>{{ category.published_courses_count }} cursos publicados</p>
```

7.6. `courses/views.py` — `MyCourseListView`

Qué reemplaza: el método `get_queryset()` actual.

```
from django.db.models import Count

class MyCourseListView(LoginRequiredMixin, ListView):
    model = Course
    template_name = "courses/my_course_list.html"
    context_object_name = "courses"

    def get_queryset(self):
        return (
            Course.objects
            .filter(academy__owner=self.request.user)
            .select_related("academy", "category", "teacher")
            .annotate(favorite_count=Count("favorites", distinct=True))
        )
```

Template `courses/my_course_list.html`:

```
<p>{{ course.favorite_count }} personas lo han guardado</p>
```

7.7. `academies/views.py` — `AcademyListView`

Qué reemplaza: el método `get_queryset()` actual.

```
from django.db.models import Avg, Count

class AcademyListView(ListView):
    model = Academy
    template_name = "academies/academy_list.html"
    context_object_name = "academies"
    paginate_by = 12

    def get_queryset(self):
        queryset = (
            Academy.objects
            .filter(is_active=True)
            .select_related("owner")
            .annotate(
                courses_count=Count("courses", distinct=True),
                teachers_count=Count("teachers", distinct=True),
                favorite_count=Count("favorites", distinct=True),
                reviews_avg=Avg("reviews__rating"),
            )
        )
        # filtros que ya tenías
        return queryset
```

Template `academies/academy_list.html`:

```
<p>{{ academy.courses_count }} cursos · {{ academy.teachers_count }} profesores</p>
<p>{{ academy.favorite_count }} personas la siguen</p>
{% if academy.reviews_avg %}
    <p>Valoración media: {{ academy.reviews_avg|floatformat:1 }}/5</p>
{% endif %}
```

7.8. `academies/views.py` — `AcademyDetailView`

Qué reemplaza: el método `get_context_data()` actual.

```
from django.db.models import Avg, Count

class AcademyDetailView(DetailView):
    model = Academy
    template_name = "academies/academy_detail.html"
    context_object_name = "academy"

    # get_queryset y get_object igual que antes

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)

        context["is_favorite"] = False
        if self.request.user.is_authenticated:
            context["is_favorite"] = AcademyFavorite.objects.filter(
                user=self.request.user,
                academy=self.object,
            ).exists()

        context["favorite_count"] = self.object.favorites.count()

        context["reviews_stats"] = self.object.reviews.aggregate(
            avg=Avg("rating"),
            total=Count("id"),
        )

        context["courses_count"] = self.object.courses.count()

        return context
```

Template `academies/academy_detail.html`:

```
<p>{{ courses_count }} cursos publicados</p>
<p>{{ favorite_count }} personas la siguen</p>
{% if reviews_stats.avg %}
    <p>Valoración media: {{ reviews_stats.avg|floatformat:1 }}/5</p>
{% endif %}
```

7.9. `academies/views.py` — `TeacherListView`

Qué reemplaza: el método `get_queryset()` actual.

```
from django.db.models import Count

class TeacherListView(ListView):
    model = Teacher
    template_name = "academies/teacher_list.html"
    context_object_name = "teachers"
    paginate_by = 12

    def get_queryset(self):
        queryset = (
            Teacher.objects
            .filter(is_active=True)
            .select_related("academy")
            .annotate(courses_count=Count("courses", distinct=True))
        )
```

```
        # filtros que ya tenías
        return queryset
```

Template `academies/teacher_list.html`:

```
<p>{{ teacher.courses_count }} cursos</p>
```

7.10. `level_tests/views.py` — `LevelTestListView`

Qué reemplaza: el método `get_queryset()` actual.

```
from django.db.models import Count

class LevelTestListView(ListView):
    model = LevelTest
    template_name = "level_tests/level_test_list.html"
    context_object_name = "level_tests"
    paginate_by = 12

    def get_queryset(self):
        queryset = (
            LevelTest.objects
            .filter(is_active=True)
            .select_related("course", "course__academy")
            .annotate(attempts_count=Count("attempts", distinct=True))
        )
        # filtros que ya tenías
        return queryset
```

Template `level_tests/level_test_list.html`:

```
<p>{{ level_test.attempts_count }} intentos</p>
```

7.11. `dashboard/views.py` — `DashboardView`

Qué reemplaza: el método `get_context_data()` actual.

```
from django.db.models import Avg, Sum

class DashboardView(LoginRequiredMixin, TemplateView):
    # get_template_names igual que antes

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        user = self.request.user

        context["total_courses"] = Course.objects.count()
        context["total_enrollments"] = Enrollment.objects.count()
        context["total_reviews"] = CourseReview.objects.count()

        context["avg_rating"] = (
            CourseReview.objects.aggregate(avg=Avg("rating"))["avg"] or 0
        )

        context["total_revenue"] = (
            Payment.objects.filter(status="paid")
            .aggregate(total=Sum("amount"))["total"] or 0
```

```
        )

        if user.is_academy:
            context["my_courses"] = Course.objects.filter(academy__owner=user)
            context["my_enrollments"] = Enrollment.objects.filter(
                convocatoria__course__academy__owner=user
            )

        elif user.is_student:
            context["my_enrollments"] = Enrollment.objects.filter(student=user)
            context["my_payments"] = Payment.objects.filter(enrollment__student=user)
            context["my_reviews"] = CourseReview.objects.filter(student=user)

        return context
```

---

8. Optimización de consultas: tabla por app

8.1. `select_related` (ForeignKey)

App	Vista	Campos
`courses`	`CourseListView`	`academy`, `category`, `teacher`
`courses`	`CourseDetailView`	`academy`, `category`, `teacher`
`courses`	`MyCourseListView`	`academy`, `category`, `teacher`
`courses`	`CategoryListView`	— (no tiene FK mostrada)
`academies`	`AcademyListView`	`owner`
`academies`	`AcademyDetailView`	`owner`
`academies`	`TeacherListView`	`academy`
`academies`	`TeacherDetailView`	`academy`
`academies`	`AcademyFavoriteListView`	`academy`, `academy__owner`
`courses`	`CourseFavoriteListView`	`course`, `course__academy`, `course__category`, `course__teacher`
`enrollments`	`EnrollmentListView`	`convocatoria`, `convocatoria__course`
`enrollments`	`AcademyEnrollmentListView`	`student`, `convocatoria`, `convocatoria__course`
`payments`	`PaymentListView`	`enrollment`, `enrollment__student`, `enrollment__convocatoria__course`
`reviews`	`CourseReviewListView`	`student`, `course`
`reviews`	`MyReviewListView`	`course`
`notifications`	`NotificationListView`	— (no tiene FK mostrada)

8.2. `prefetch_related` (1:N y M:N)

App	Vista	Campos
`courses`	`CourseDetailView`	`images`, `convocatorias`
`courses`	`CourseListView`	`images` (con `Prefetch` ordenado por `position`)
`courses`	`CourseFavoriteListView`	`course__images`
`academies`	`AcademyDetailView`	`teachers`, `courses`
`academies`	`AcademyListView`	`courses`

8.3. `Prefetch`

Solo se aplica en `CourseListView`, para cargar imágenes ordenadas:

```
Prefetch(
    "images",
    queryset=CourseImage.objects.order_by("position"),
)
```

8.4. `exists()`, `count()`, `update()`, `bulk_create()`, `explain()`

- `exists()`: ya se usa en `toggle_course_favorite` y `toggle_academy_favorite`.
- `count()`: se usa en el detalle de cursos y academias en lugar de `len()`.
- `update()`: ya se usa en `CourseDetailView` y `AcademyDetailView` con `F("views_count") + 1`.
- `bulk_create()`: no se aplica en esta sesión, se documenta para futuras.
- `explain()`: se documenta en el registro de sesión como herramienta de diagnóstico.

---

9. Transacciones: qué vistas y por qué

9.1. Vistas que ya tienen `transaction.atomic`

App	Vista	Por qué
`courses`	`CourseCreateView`	Crea curso + varias `CourseImage` a la vez
`courses`	`toggle_course_favorite`	Crea o borra `CourseFavorite`
`academies`	`toggle_academy_favorite`	Crea o borra `AcademyFavorite`

9.2. Vistas que se añaden

Ninguna. El resto de vistas no crean varios objetos a la vez.

9.3. `select_for_update`

No se integra en vistas en esta sesión. Se deja documentado en los servicios (sección 10) para usarse en la Sesión 9.

---

10. Servicios: creación de archivos y tests

10.1. `enrollments/services.py`

Archivo: `enrollments/services.py` (crear).

Qué hace: reserva de plaza con transacción y bloqueo.

No se integra aún en las vistas. Se usará en la Sesión 9.

```
from django.db import transaction

from courses.models import Convocatoria
from enrollments.models import Enrollment, EnrollmentStatus


@transaction.atomic
def reserve_place(student, convocatoria_id):
    convocatoria = (
        Convocatoria.objects
        .select_for_update()
        .get(pk=convocatoria_id)
    )

    if not convocatoria.is_active:
        raise ValueError("La convocatoria no está activa.")

    if convocatoria.is_full:
        raise ValueError("La convocatoria está llena.")

    if Enrollment.objects.filter(
        student=student,
        convocatoria=convocatoria,
    ).exists():
        raise ValueError("Ya tienes una inscripción en esta convocatoria.")

    enrollment = Enrollment.objects.create(
        student=student,
        convocatoria=convocatoria,
        status=EnrollmentStatus.RESERVED,
    )

    convocatoria.reserved_places += 1
    convocatoria.save(update_fields=["reserved_places"])

    return enrollment
```

10.2. `courses/services.py`

Archivo: `courses/services.py` (crear).

Qué hace: archivado con comprobación de propietario.

No se integra aún en las vistas. Se usará cuando haya una vista de "archivar curso".

```
from django.db import transaction

from courses.models import Course, CourseStatus


@transaction.atomic
def archive_course(course_id, user):
    course = (
        Course.objects
        .select_for_update()
        .get(pk=course_id)
    )

    if course.academy.owner != user:
        raise ValueError("No tienes permiso para archivar este curso.")

    course.status = CourseStatus.ARCHIVED
    course.save(update_fields=["status"])
    return course
```

10.3. `academies/services.py`

Archivo: `academies/services.py` (crear).

Qué hace: verificación de academia, solo por staff.

No se integra aún en las vistas. Se usará cuando haya un panel de moderación.

```
from django.db import transaction

from academies.models import Academy


@transaction.atomic
def verify_academy(academy_id, staff_user):
    if not staff_user.is_staff:
        raise ValueError("Solo el staff puede verificar academias.")

    academy = (
        Academy.objects
        .select_for_update()
        .get(pk=academy_id)
    )

    academy.is_verified = True
    academy.save(update_fields=["is_verified"])
    return academy
```

---

11. Reorganización de tests por app

11.1. Estructura final

Cada app pasa de tener `tests.py` a tener:

```
<app>/
    tests/
        __init__.py
        test_models.py
        test_views.py
        test_services.py
        test_queries.py
```

11.2. Qué va en cada archivo

Archivo	Contenido
`__init__.py`	Vacío
`test_models.py`	Tests de modelos y constraints
`test_views.py`	Tests de vistas, permisos y plantillas
`test_services.py`	Tests de servicios con transacciones (solo en apps con `services.py`)
`test_queries.py`	Tests de `annotate`, `aggregate`, filtros ORM

11.3. Qué hacer con el `tests.py` actual

1. Crear la carpeta `tests/` en cada app.
2. Crear `__init__.py` vacío.
3. Repartir el contenido del `tests.py` actual entre los archivos que corresponda.
4. **Borrar el `tests.py` original** después de mover el contenido.

11.4. Cómo repartir el contenido de cada app

App	test_models.py	test_views.py	test_services.py	test_queries.py
accounts	UserModelTests	LoginTests, ProfileTests, RegisterTests, UserProfileViewTests	—	—
academies	AcademyModelTests, TeacherModelTests	AcademyViewTests, TeacherViewTests	VerifyAcademyTests	—
courses	CourseModelTests, CategoryModelTests, ConvocatoriaModelTests, CourseFavoriteModelTests	CourseViewTests, CourseFavoriteTests, CourseSearchTests	ArchiveCourseTests	CourseQueriesTests
enrollments	EnrollmentModelTests, WaitlistEntryModelTests, EnrollmentAnswerModelTests, EnrollmentDocumentModelTests	EnrollmentViewTests	ReservePlaceTests	—

App	test_models.py	test_views.py	test_services.py	test_queries.py
reviews	CourseReviewModelTests, AcademyReviewModelTests	CourseReviewViewTests, AcademyReviewViewTests	—	—
payments	DiscountModelTests, PaymentModelTests, ScholarshipModelTests	DiscountViewTests, PaymentViewTests, ScholarshipViewTests	—	—
talent	CompetenceModelTests, TalentPassportModelTests, PassportEntryModelTests	PassportViewTests	—	—
level_tests	LevelTestModelTests, LevelTestAttemptModelTests	LevelTestViewTests	—	—
notifications	NotificationModelTests	NotificationViewTests	—	—
evaluations	EvaluationModelTests	EvaluationViewTests	—	—
core	ContactMessageModelTests	ContactMessageViewTests	—	—
search	—	SearchTests	—	—
dashboard	—	DashboardTests	—	—

11.5. Ejemplo completo de `courses/tests/`

`courses/tests/__init__.py` (vacío).

`courses/tests/test_models.py`

```
from decimal import Decimal

from django.contrib.auth import get_user_model
from django.db import IntegrityError
from django.test import TestCase

from academies.models import Academy
from courses.models import (
    Category,
    Convocatoria,
    Course,
    CourseFavorite,
    CourseStatus,
)

User = get_user_model()


class ConvocatoriaConstraintTests(TestCase):

    def setUp(self):
        self.user = User.objects.create_user(
            username="englishnow",
```

```
            email="info@englishnow.com",
            password="TestPassword123!",
            is_academy=True,
        )
        self.academy = Academy.objects.create(
            owner=self.user,
            name="English Now",
            slug="english-now",
        )
        self.category = Category.objects.create(
            name="Idiomas",
            slug="idiomas",
        )
        self.course = Course.objects.create(
            academy=self.academy,
            category=self.category,
            title="Inglés B2",
            slug="ingles-b2",
            description="...",
            status=CourseStatus.PUBLISHED,
        )

    def test_price_must_be_positive(self):
        with self.assertRaises(IntegrityError):
            Convocatoria.objects.create(
                course=self.course,
                start_date="2026-07-01",
                price=Decimal("-10.00"),
                total_places=15,
            )

    def test_deposit_must_be_positive_or_zero(self):
        with self.assertRaises(IntegrityError):
            Convocatoria.objects.create(
                course=self.course,
                start_date="2026-07-01",
                end_date="2026-09-30",
                price=Decimal("480.00"),
                deposit=Decimal("-5.00"),
                total_places=15,
            )


class CourseFavoriteConstraintTests(TestCase):

    def setUp(self):
        self.user = User.objects.create_user(
            username="englishnow",
            email="info@englishnow.com",
            password="TestPassword123!",
            is_academy=True,
        )
        self.student = User.objects.create_user(
            username="laura",
            email="laura@example.com",
            password="TestPassword123!",
            is_student=True,
        )
        self.academy = Academy.objects.create(
            owner=self.user,
            name="English Now",
```

```
            slug="english-now",
        )
        self.category = Category.objects.create(
            name="Idiomas",
            slug="idiomas",
        )
        self.course = Course.objects.create(
            academy=self.academy,
            category=self.category,
            title="Inglés B2",
            slug="ingles-b2",
            description="...",
            status=CourseStatus.PUBLISHED,
        )

    def test_user_cannot_favorite_course_twice(self):
        CourseFavorite.objects.create(
            user=self.student,
            course=self.course,
        )
        with self.assertRaises(IntegrityError):
            CourseFavorite.objects.create(
                user=self.student,
                course=self.course,
            )
```

courses/tests/test_views.py

```
from decimal import Decimal

from django.contrib.auth import get_user_model
from django.test import TestCase
from django.urls import reverse

from academies.models import Academy
from courses.models import (
    Category,
    Convocatoria,
    Course,
    CourseFavorite,
    CourseStatus,
)

User = get_user_model()


class CourseViewTests(TestCase):

    def setUp(self):
        self.academy_user = User.objects.create_user(
            username="englishnow",
            email="info@englishnow.com",
            password="TestPassword123!",
            is_academy=True,
        )
        self.other_academy_user = User.objects.create_user(
            username="codeacademy",
            email="info@codeacademy.com",
            password="TestPassword123!",
            is_academy=True,
        )
```

```
        self.academy = Academy.objects.create(
            owner=self.academy_user,
            name="English Now",
            slug="english-now",
        )
        self.category = Category.objects.create(
            name="Idiomas",
            slug="idiomas",
        )
        self.course = Course.objects.create(
            academy=self.academy,
            category=self.category,
            title="Inglés B2",
            slug="ingles-b2",
            description="...",
            status=CourseStatus.PUBLISHED,
        )

    def test_list_is_public(self):
        response = self.client.get(reverse("courses:list"))
        self.assertEqual(response.status_code, 200)

    def test_detail_is_public(self):
        response = self.client.get(
            reverse("courses:detail", kwargs={"pk": self.course.pk, "slug": "ingles-b2"})
        )
        self.assertEqual(response.status_code, 200)

    def test_other_user_cannot_edit(self):
        self.client.force_login(self.other_academy_user)
        response = self.client.get(
            reverse("courses:update", kwargs={"pk": self.course.pk})
        )
        self.assertEqual(response.status_code, 403)

    def test_owner_can_edit(self):
        self.client.force_login(self.academy_user)
        response = self.client.get(
            reverse("courses:update", kwargs={"pk": self.course.pk})
        )
        self.assertEqual(response.status_code, 200)


class CourseFavoriteTests(TestCase):

    def setUp(self):
        self.academy_user = User.objects.create_user(
            username="englishnow",
            email="info@englishnow.com",
            password="TestPassword123!",
            is_academy=True,
        )
        self.student = User.objects.create_user(
            username="laura",
            email="laura@example.com",
            password="TestPassword123!",
            is_student=True,
        )
        self.academy = Academy.objects.create(
            owner=self.academy_user,
            name="English Now",
```

```
            slug="english-now",
        )
        self.category = Category.objects.create(name="Idiomas", slug="idiomas")
        self.course = Course.objects.create(
            academy=self.academy,
            category=self.category,
            title="Inglés B2",
            slug="ingles-b2",
            description="...",
            status=CourseStatus.PUBLISHED,
        )

    def test_student_can_add_favorite(self):
        self.client.force_login(self.student)
        self.client.post(
            reverse("courses:toggle_favorite", kwargs={"pk": self.course.pk}),
        )
        self.assertTrue(
            CourseFavorite.objects.filter(
                user=self.student,
                course=self.course,
            ).exists()
        )

    def test_favorite_requires_login(self):
        response = self.client.post(
            reverse("courses:toggle_favorite", kwargs={"pk": self.course.pk}),
        )
        self.assertEqual(response.status_code, 302)
```

courses/tests/test_services.py

```
from django.contrib.auth import get_user_model
from django.test import TestCase

from academies.models import Academy
from courses.models import Category, Course, CourseStatus
from courses.services import archive_course

User = get_user_model()


class ArchiveCourseTests(TestCase):

    def setUp(self):
        self.owner = User.objects.create_user(
            username="englishnow",
            email="info@englishnow.com",
            password="TestPassword123!",
            is_academy=True,
        )
        self.other = User.objects.create_user(
            username="codeacademy",
            email="info@codeacademy.com",
            password="TestPassword123!",
            is_academy=True,
        )
        self.academy = Academy.objects.create(
            owner=self.owner,
            name="English Now",
            slug="english-now",
```

```
        )
        self.category = Category.objects.create(name="Idiomas", slug="idiomas")
        self.course = Course.objects.create(
            academy=self.academy,
            category=self.category,
            title="Inglés B2",
            slug="ingles-b2",
            description="...",
            status=CourseStatus.PUBLISHED,
        )

    def test_owner_can_archive_course(self):
        archive_course(self.course.pk, self.owner)
        self.course.refresh_from_db()
        self.assertEqual(self.course.status, CourseStatus.ARCHIVED)

    def test_other_user_cannot_archive_course(self):
        with self.assertRaises(ValueError):
            archive_course(self.course.pk, self.other)
```

courses/tests/test_queries.py

```
from django.contrib.auth import get_user_model
from django.db.models import Avg, Count
from django.test import TestCase

from academies.models import Academy
from courses.models import Category, Course, CourseStatus

User = get_user_model()


class CourseQueriesTests(TestCase):

    def setUp(self):
        self.user = User.objects.create_user(
            username="englishnow",
            email="info@englishnow.com",
            password="TestPassword123!",
            is_academy=True,
        )
        self.academy = Academy.objects.create(
            owner=self.user,
            name="English Now",
            slug="english-now",
        )
        self.category = Category.objects.create(name="Idiomas", slug="idiomas")
        for i in range(3):
            Course.objects.create(
                academy=self.academy,
                category=self.category,
                title=f"Curso {i}",
                slug=f"curso-{i}",
                description="...",
                status=CourseStatus.PUBLISHED,
            )

    def test_courses_count_by_category(self):
        result = Category.objects.annotate(
            courses_count=Count("courses"),
        ).get(pk=self.category.pk)
```

```
        self.assertEqual(result.courses_count, 3)

    def test_avg_duration_hours(self):
        result = Course.objects.aggregate(
            avg=Avg("duration_hours"),
        )
        self.assertIsNotNone(result["avg"])
```

11.6. Repetir el patrón en todas las apps

Cada app sigue el mismo esquema. La tabla de la sección 11.4 dice qué contenido va en cada archivo.

---

12. Backup básico

12.1. Añadir a `.gitignore`

Añadir al final de `.gitignore`:

```
*.sql
backups/
```

12.2. Crear backup

```
pg_dump -U academyhub_user -d academyhub -h localhost -p 5432 > academyhub_backup.sql
```

12.3. Restaurar backup

```
psql -U academyhub_user -d academyhub -h localhost -p 5432 < academyhub_backup.sql
```

12.4. Comprobar que `.env` no está en Git

```
git status
```

`.env` no debe aparecer. Si aparece, revisar `.gitignore`:

```
.env
```

---

13. Checklist global

PostgreSQL

- [x] PostgreSQL instalado.
- [x] `psql --version` funciona.
- [x] Usuario `academyhub_user` creado.
- [x] Base `academyhub` creada.
- [x] `psycopg[binary]` instalado.

- [x] Variables de entorno en `.env`.
- [x] Django conectado a PostgreSQL.
- [x] Migraciones ejecutadas.
- [x] Superusuario creado.

Integridad

- [x] `Academy.views_count >= 0`.
- [x] `Course.views_count >= 0`.
- [x] `Course.duration_hours >= 0`.
- [x] `Convocatoria.price > 0`.
- [x] `Convocatoria.deposit >= 0`.
- [x] `Convocatoria.reserved_places >= 0`.
- [x] `Convocatoria.enrolled_places >= 0`.
- [x] `Enrollment` único por alumno y convocatoria.
- [x] `WaitlistEntry` único por alumno y convocatoria.
- [x] `CourseReview` único por alumno y curso.
- [x] `AcademyReview` único por alumno y academia.
- [x] `rating` entre 1 y *5*.
- [x] `PassportEntry.level` entre 1 y *5*.
- [x] `Payment.amount >= 0`.
- [x] `Scholarship.amount >= 0`.
- [x] `Discount.value >= 0`.
- [x] `LevelTest.passing_score` entre 0 y 100.
- [x] `LevelTestAttempt.score` entre 0 y 100.

Índices

- [x] Índices simples mantenidos.
- [x] Índices compuestos nuevos.
- [x] Índice `(user, is_read)` en notificaciones.
- [x] Índice `(passport, -obtained_at)` en pasaporte.
- [x] Índice `(student, -started_at)` en intentos de prueba.
- [x] Índices `(status, -created_at)` y `(status, category)` en cursos.

ORM

- [x] `filter`, `exclude`, `Q`, `F`.
- [x] `values`, `values_list`.
- [x] `annotate`, `aggregate`.
- [x] `Count`, `Avg`, `Min`, `Max`, `Sum`.

Optimización

- [x] `select_related` en todas las vistas con `ForeignKey`.
- [x] `prefetch_related` en todas las vistas con `1:N` y `M:N`.
- [x] `Prefetch` en `CourseListView`.
- [x] `exists()` en las comprobaciones booleanas.
- [x] `count()` en lugar de `len()`.
- [x] `update()` documentado.
- [x] `bulk_create()` documentado.

- [x] `explain()` documentado.

Transacciones

- [x] `transaction.atomic` en `CourseCreateView`.
- [x] `transaction.atomic` en `toggle_course_favorite`.
- [x] `transaction.atomic` en `toggle_academy_favorite`.
- [x] `services.py` en `enrollments` con `reserve_place`.
- [x] `services.py` en `courses` con `archive_course`.
- [x] `services.py` en `academies` con `verify_academy`.
- [x] `select_for_update` documentado y testeado.
- [x] Race condition documentada.

Tests

- [x] Tests de constraints.
- [x] Tests de servicios.
- [x] Tests de `annotate` y `aggregate`.
- [x] Tests ejecutados contra PostgreSQL.
- [x] Tests reorganizados en carpetas por app.

---

14. Registro de sesión

```
PROYECTO: Academy Hub
SESIÓN: 07
FASE: Datos y optimización
TEMA: PostgreSQL, ORM avanzado, constraints, índices y transacciones
FECHA:
HORA DE ENTRADA:
HORA DE SALIDA:
DURACIÓN:

OBJETIVO:
Migrar Academy Hub desde SQLite a PostgreSQL y aprender técnicas de
integridad, optimización y concurrencia.

TEORÍA:
- PostgreSQL
- ORM
- SQL
- Constraints
- Índices simples y compuestos
- N+1
- select_related
- prefetch_related
- Prefetch
- annotate
- aggregate
- Count, Avg, Min, Max, Sum
- values, values_list
```

- transacciones
- atomicidad
- select_for_update
- race conditions
- EXPLAIN

IMPLEMENTACIÓN:
- Base PostgreSQL
- Usuario PostgreSQL
- psycopg
- Variables de entorno
- Configuración Django
- Migraciones
- CheckConstraint
- Índices simples y compuestos
- Consultas agregadas
- Servicios con transacciones
- Reorganización de tests

ARCHIVOS MODIFICADOS:
.env
.env.example
requirements.txt
.gitignore
config/settings/development.py
config/settings/production.py
config/settings/testing.py

accounts/models.py
academies/models.py
courses/models.py
enrollments/models.py
reviews/models.py
payments/models.py
talent/models.py
level_tests/models.py
notifications/models.py
evaluations/models.py

enrollments/services.py
courses/services.py
academies/services.py

courses/views.py
academies/views.py
level_tests/views.py
dashboard/views.py

<app>/tests/__init__.py
<app>/tests/test_models.py
<app>/tests/test_views.py
<app>/tests/test_services.py

```
<app>/tests/test_queries.py

BASE DE DATOS:
ANTES: SQLite
DESPUÉS: PostgreSQL

TESTS:
- constraints
- servicios
- annotate/aggregate
- explain

PROBLEMAS:
SOLUCIONES:

COMMIT:
feat: migrate to PostgreSQL and optimize ORM with constraints and services

ESTADO:
[ ] Pendiente
[ ] En progreso
[ ] Completada

SIGUIENTE SESIÓN:
Django REST Framework: serializers, APIView, generic views, ViewSets,
routers, permissions, pagination, filtering y documentación de la API.
```

---

15. Commit recomendado

```
git status
git add .
git commit -m "feat: migrate to PostgreSQL and optimize ORM with constraints and services"
```

Aviso: las migraciones nuevas de cada app deben ir en el mismo commit. Comprobar antes:

```
git status
```

Deberían aparecer:

- `<app>/migrations/000X_....py` de cada app.
- `.env` NO debe aparecer.
- `*.sql` y `backups/` NO deben aparecer.